Matemáticas Aplicadas para Ciencia de los Datos

Clase 1: Representación y Operaciones con Vectores, Matrices y Tensores, y algo de distancias

Diego Fonseca[1]

[1]Universidad EAFIT, Medellín, Colombia

Febrero 13, 2026

Introducción y Formalidades

Introducción: Pacto Pedagógico: Principios

Como miembros de la comunidad académica de la Universidad EAFIT, nos comprometemos a actuar con integridad, adhiriéndonos a los más altos estándares éticos y morales.

Nuestra interacción se basará en los siguientes pilares:

- **Respeto**
- **Tolerancia**
- **Honradez**
- **Compromiso**

Introducción: Pacto Pedagógico: Logística del Curso

Plataforma del curso:
Toda la información, materiales y anuncios se gestionarán a través de **EAFIT Interactiva**.

Evaluaciones y Fechas:
El esquema de evaluación se detalla en el programa del curso.

- 3 Exámenes: **75 %** (25 % cada uno).
- 1 Práctica en grupo: **25 %**.

Libro Guía Principal:
Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press.

Disponible en línea

Introducción: Pacto Pedagógico: Orientaciones

Responsabilidad Compartida:
El éxito del curso depende del compromiso tanto del **profesor** como de los **estudiantes**. Se espera una participación activa y una preparación constante.

Orientaciones Clave:

- Se recomienda un mínimo de **seis horas de trabajo autónomo** por semana.
- La asistencia a las clases presenciales es fundamental para la comprensión de los temas.
- La **evaluación a la docencia** es una parte importante del proceso de mejora continua y su realización es obligatoria.
- Es responsabilidad del estudiante revisar periódicamente los canales oficiales: **EAFIT Interactiva** y **correo institucional**.
- Las prácticas se realizarán en equipos de **máximo dos estudiantes**.

Representación de Datos mediante vectores, matrices y tensores

Tema: Representación de Datos: Motivación

En la ciencia de datos, nos enfrentamos a información de gran variedad y complejidad: desde simples tablas numéricas hasta imágenes, videos o redes de interacciones.

Para analizar estos datos de manera sistemática, necesitamos un **lenguaje matemático** que nos permita estructurarlos y manipularlos. El álgebra lineal nos proporciona este lenguaje.

Comenzaremos con los bloques de construcción fundamentales: los **vectores** y las **matrices**, para luego generalizar estos conceptos a **tensores**.

El concepto más básico de contenedor de datos es el **vector**.

Definición (Vector)

Un **vector** es un conjunto **ordenado** de números, llamados **componentes**. El orden es crucial: el vector $(1, 2)$ es distinto de $(2, 1)$.

Vector Renglón:
Los componentes se escriben horizontalmente.

$$\mathbf{v} = (x_1, x_2, \ldots, x_n)$$

Vector Columna:
Los componentes se escriben verticalmente.

$$\mathbf{u} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

Si un vector tiene n componentes, decimos que su **dimensión** es n. El conjunto de todos los vectores de dimensión n con componentes reales se denota como $\mathbb{R}^n$.

Ejemplo

1. $(3, 6)$ es un vector renglón de dimensión 2, un elemento de $\mathbb{R}^2$.
2. $\begin{pmatrix} 2 \\ -1 \\ 5 \end{pmatrix}$ es un vector columna de dimensión 3, un elemento de $\mathbb{R}^3$.
3. $(2, -1, 0, 3)$ es un vector renglón de dimensión 4.
4. $\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$ es un vector columna de dimensión 3. Se conoce como el **vector cero** en $\mathbb{R}^3$.

Observación

En ciencia de datos, un vector puede representar una **observación** o **instancia**. Por ejemplo, si medimos la altura, peso y edad de una persona, podríamos representarla con el vector $(175, 70, 30)$.

Si un vector es una lista de números, una **matriz** es una tabla o un arreglo rectangular de números.

Definición (Matriz)

Una **matriz** $\mathbf{A}$ es un arreglo bidimensional de números organizados en **filas** (renglones) y **columnas**.

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}$$

- Si $\mathbf{A}$ tiene m filas y n columnas, decimos que su **tamaño** o **dimensión** es $m \times n$.
- El número a_{ij} es la **componente** en la fila i y la columna j.
- Si $m = n$, la matriz es **cuadrada**.

Ejemplo

1. $\begin{pmatrix} 1 & 3 \\ -1 & 2 \end{pmatrix}$ es una matriz cuadrada de tamaño 2×2.
2. $\begin{pmatrix} -1 & 3 \\ 4 & 0 \\ 2 & -5 \end{pmatrix}$ es una matriz de tamaño 3×2.
3. $\begin{pmatrix} 1 & -2 & 3 \\ 4 & 0 & -5 \end{pmatrix}$ es una matriz de tamaño 2×3.
4. Una matriz donde todas sus componentes son cero es una **matriz cero**.

Definición (Igualdad de Matrices)

Dos matrices $\mathbf{A} = (a_{ij})$ y $\mathbf{B} = (b_{ij})$ son **iguales** si y solo si:

1. Tienen el mismo tamaño $(m \times n)$.
2. Sus componentes correspondientes son idénticas, es decir, $a_{ij} = b_{ij}$ para todo i, j.

Tema: Generalización: ¿Qué es un Tensor?

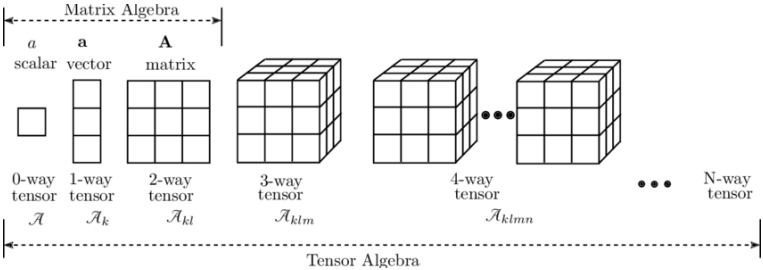
Hemos visto que los datos se pueden organizar en listas (vectores) o tablas (matrices). Pero, ¿qué pasa si la estructura de los datos es más compleja?

En el contexto de la ciencia de datos, un **tensor** es simplemente una **generalización** de los escalares, vectores y matrices a un número arbitrario de dimensiones.

Jerarquía de Tensores

- **Orden 0:** Un escalar (un solo número). $s \in \mathbb{R}$.
- **Orden 1:** Un vector (un arreglo 1D). $\mathbf{v} \in \mathbb{R}^n$.
- **Orden 2:** Una matriz (un arreglo 2D). $\mathbf{M} \in \mathbb{R}^{m \times n}$.
- **Orden N:** Un **tensor** (un arreglo ND). $\mathcal{T} \in \mathbb{R}^{d_1 \times d_2 \times \cdots \times d_N}$.

La forma más intuitiva de entender un tensor es como un **contenedor de datos multidimensional**.



Observación (Definición Formal vs. Práctica)

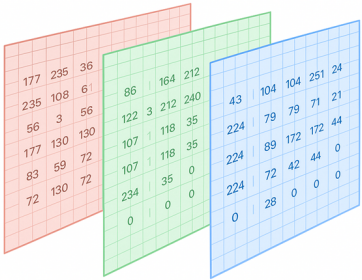
En matemáticas puras, un tensor es un objeto geométrico o una función multilineal. Para nuestros propósitos en ciencia de datos, la visión de un **array multidimensional** es mucho más útil y es la que adoptan librerías como `NumPy`, `PyTorch` y `TensorFlow`.

Tema: Generalización: Ejemplo: Una Imagen como Tensor

Un ejemplo perfecto de un tensor de orden 3 es una **imagen a color**. Se puede representar como un tensor de dimensiones $(H \times W \times C)$:

- **H**: Altura de la imagen en píxeles.
- **W**: Ancho de la imagen en píxeles.
- **C**: Número de canales de color (usualmente 3 para RGB: Rojo, Verde, Azul).

Imagina este tensor como un "cubo" de números, donde cada "rebanada" del cubo es una matriz que representa la intensidad de uno de los colores.



Tema: Aplicaciones: Estructuras de Datos Comunes

La forma en que organizamos los datos en el mundo real casi siempre se puede mapear a una estructura matricial o tensorial. Veamos tres casos comunes:

Tablas
(Datos transversales)

Series de Tiempo
(Datos a lo largo del tiempo)

Datos de Panel
(Múltiples series de tiempo)

Tema: Aplicaciones: Tablas y Series de Tiempo

Tablas (Datos Transversales)

Una tabla de datos, como una hoja de Excel, es una **matriz**.

- Cada **fila** representa una **observación** (un cliente, un producto, un día).
- Cada **columna** representa una **característica** o **variable** (edad, precio, temperatura).

Series de Tiempo

Una serie de tiempo es una secuencia de observaciones tomadas en intervalos de tiempo sucesivos. Se puede representar como un **vector columna** (si hay una sola variable) o una **matriz** (si hay múltiples variables).

$$\text{Precio acción Apple} = \begin{pmatrix} p(t_1) \\ p(t_2) \\ \vdots \\ p(t_N) \end{pmatrix} \in \mathbb{R}^{N \times 1}$$

Tema: Aplicaciones: Datos de Panel

Los **datos de panel** (o datos longitudinales) combinan la dimensión de sección transversal y la de series de tiempo. Siguen a **múltiples entidades** a lo largo del **tiempo**.

Se representan naturalmente como una matriz $\mathbf{X} \in \mathbb{R}^{N \times T}$, donde:

- N es el número de entidades (empresas, países, pacientes).
- T es el número de períodos de tiempo.

Ejemplo (Portafolio de Acciones)

Queremos analizar el precio diario de 100 acciones durante el último año (252 días hábiles). Nuestros datos forman una matriz de tamaño 100×252.

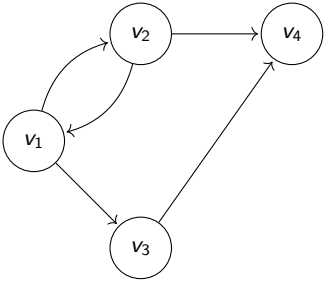
- La fila i es la serie de tiempo de la acción i.
- La columna j es el vector de precios de todas las acciones en el día j.

Un **grafo** es una estructura matemática usada para modelar relaciones entre objetos. Consiste en **vértices** (nodos) y **aristas** (conexiones).

Definición (Grafo)

Un grafo G es un par (V, E) donde V es un conjunto de vértices y E es un conjunto de aristas que conectan pares de vértices.

Las matrices nos permiten **representar algebraicamente** la estructura de un grafo, lo que facilita su análisis computacional.



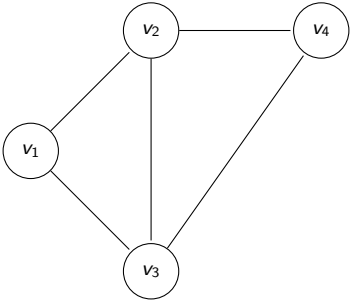
Matriz de Adyacencia (A):
Es una matriz cuadrada donde $A_{ij} = 1$ si hay una arista entre el vértice i y el vértice j, y $A_{ij} = 0$ en caso contrario. Es la representación más directa de la conectividad.

Matriz de Grado (D):
Es una matriz **diagonal** donde el elemento D_{ii} contiene el **grado** del vértice i (el número de aristas conectadas a él).

Matriz Laplaciana (L):
Se define como $\mathbf{L} = \mathbf{D} - \mathbf{A}$. Es fundamental en análisis espectral de grafos, clustering y detección de comunidades.

Tema: Grafos: Ejemplo de Representación

Para el siguiente grafo no dirigido:



Matriz de adyacencia A:

$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

Matriz de grado D:

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}$$

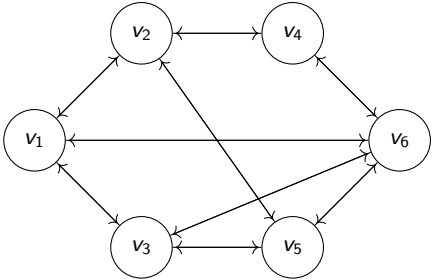
Tema: Grafos: Ejemplo: Redes Sociales

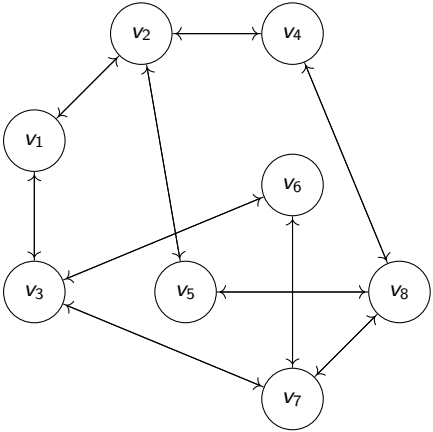
En una red social, podemos modelar a los **usuarios como vértices** y las **amistades como aristas**.

- La **matriz de adyacencia** nos dice quién es amigo de quién.
- La **matriz de grado** nos da el número de amigos de cada persona (una medida simple de popularidad).
- El análisis de la **matriz Laplaciana** puede revelar comunidades (grupos de amigos densamente conectados) o identificar a los usuarios más influyentes para la difusión de información.

Actividad

Para cada uno de los siguientes grafos, determine su **matriz de adyacencia** y su **matriz de grado**. Asuma que son no dirigidos.





Actividad

Dada la siguiente matriz de adyacencia **A** para un grafo de 6 vértices:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}.$$

1. Dibuje el grafo no dirigido correspondiente.
2. Calcule el grado de cada vértice. ¿Cuál es el vértice con mayor conectividad?

Tema: Aplicaciones: Matrices de Frecuencia de Términos

En Procesamiento de Lenguaje Natural (PLN), necesitamos convertir texto en números. La forma más básica de hacerlo es con una **Matriz de Frecuencia de Términos** (o Term-Document Matrix).

Definición

Dado un conjunto de n documentos y un vocabulario de m términos únicos, la matriz de frecuencia de términos $\mathbf{F}$ es una matriz de tamaño $m \times n$ donde la entrada F_{ij} es el **número de veces que el término i aparece en el documento j**.

- Las **filas** representan las palabras del vocabulario.
- Las **columnas** representan los documentos.

Ejemplo

Supongamos que nuestro corpus de texto tiene 3 documentos:

- d_1: "el perro come rápido, el gato come lento"
- d_2: "el gato juega con el perro"
- d_3: "el niño juega solo"

Vocabulario (simplificado): {perro, come, gato, juega}

Matriz de frecuencia de términos F:

	d_1	d_2	d_3
perro	1	1	0
come	2	0	0
gato	1	1	0
juega	0	1	1

$$\Longrightarrow \mathbf{F} = \begin{pmatrix} 1 & 1 & 0 \\ 2 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

Esta matriz es ahora una representación numérica del corpus que puede ser usada como entrada para un modelo de Machine Learning.

Como vimos en el ejemplo del tensor, las matrices son la forma nativa de representar imágenes digitales.

- Una **imagen en escala de grises** es una única matriz $m \times n$, donde cada elemento representa la intensidad de un píxel (e.g., 0=negro, 255=blanco).
- Una **imagen a color (RGB)** se representa con 3 matrices ($m \times n$), una para cada canal de color (Rojo, Verde, Azul), o como un tensor $m \times n \times 3$.

Ejemplo en Python con NumPy:

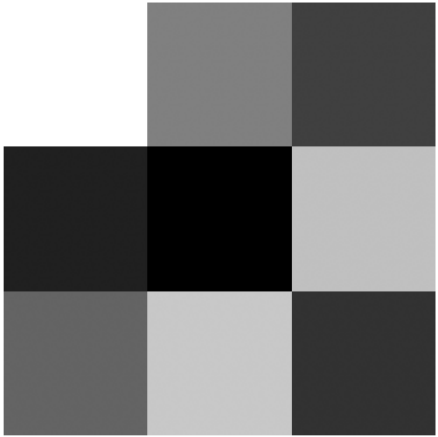
```python
from PIL import Image
import numpy as np

# Cargar una imagen y convertirla a un array de NumPy
img = Image.open('eafit_logo.png').convert('L') # 'L' para escala de grises
img_matrix = np.array(img)

# img_matrix es ahora una matriz 2D que podemos manipular
print(img_matrix.shape)
print(img_matrix)
```

Actividad

La siguiente imagen en escala de grises es generada a partir de la matriz que esta al lado.



$$K = \begin{bmatrix} 255 & 128 & 64 \\ 32 & 0 & 192 \\ 100 & 200 & 50 \end{bmatrix}$$

Actividad

Crea una imagen en escala de grises a partir de la siguiente matriz. ¿Qué figura o patrón esperas ver?

$$\mathbf{I} = \begin{pmatrix} 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 255 & 128 & 128 & 255 & 0 \\ 255 & 128 & 64 & 64 & 128 & 255 \\ 255 & 128 & 64 & 64 & 128 & 255 \\ 0 & 255 & 128 & 128 & 255 & 0 \\ 0 & 0 & 255 & 255 & 0 & 0 \end{pmatrix}$$

Observación

Las operaciones matriciales son la base de casi todo el procesamiento de imágenes: aplicar filtros (convolución), rotar, escalar, detectar bordes, comprimir (SVD) y son el núcleo de las Redes Neuronales Convolucionales (CNNs) para visión por computador.

Operaciones con Vectores y Matrices

Tema: Operaciones Básicas: Introducción

Una vez que sabemos cómo representar datos con matrices y vectores, el siguiente paso es aprender a **manipularlos**. Las operaciones algebraicas son los "verbos" que nos permiten transformar, combinar y analizar nuestros datos.

En esta sección, cubriremos las operaciones fundamentales:

- Suma y Multiplicación por un escalar.
- La fundamental (y no tan intuitiva) **multiplicación de matrices**.
- Operaciones especiales como la **transposición** y la **inversión**.

Dominar estas operaciones es la base para entender desde la regresión lineal hasta el funcionamiento interno de las redes neuronales.

Definición (Suma de Matrices)

Sean $\mathbf{A}$ y $\mathbf{B}$ dos matrices **del mismo tamaño** $m \times n$. Su **suma**, denotada como $\mathbf{A} + \mathbf{B}$, es la matriz de tamaño $m \times n$ que se obtiene sumando las componentes correspondientes:

$$(\mathbf{A} + \mathbf{B})_{ij} = a_{ij} + b_{ij}$$

Condición Fundamental

La suma de matrices solo está definida para matrices con **exactamente las mismas dimensiones**. No se puede sumar una matriz de 2×3 con una de 3×2.

Ejemplo

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} + \begin{pmatrix} 0 & -1 \\ 5 & 6 \end{pmatrix} = \begin{pmatrix} 1+0 & 2-1 \\ 3+5 & 4+6 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 8 & 10 \end{pmatrix}$$

Definición (Multiplicación por un Escalar)

Sea $\mathbf{A}$ una matriz de tamaño $m \times n$ y λ un número real (un **escalar**). El producto $\lambda\mathbf{A}$ es la matriz de tamaño $m \times n$ que se obtiene multiplicando **cada componente** de $\mathbf{A}$ por λ:

$$(\lambda\mathbf{A})_{ij} = \lambda \cdot a_{ij}$$

Esta operación "escala" la matriz, aumentando o disminuyendo la magnitud de todos sus elementos de manera uniforme.

Ejemplo

Sea $\lambda = 3$ y $\mathbf{A} = \begin{pmatrix} 2 & -1 \\ 0 & 4 \end{pmatrix}$.

$$3\mathbf{A} = 3\begin{pmatrix} 2 & -1 \\ 0 & 4 \end{pmatrix} = \begin{pmatrix} 3 \cdot 2 & 3 \cdot (-1) \\ 3 \cdot 0 & 3 \cdot 4 \end{pmatrix} = \begin{pmatrix} 6 & -3 \\ 0 & 12 \end{pmatrix}$$

Tema: Multiplicación de Matrices: Definición

La multiplicación de matrices es la operación más importante y menos intuitiva. No es una operación elemento a elemento.

Definición (Multiplicación de Matrices)

Sea **A** una matriz de tamaño $m \times \boldsymbol{n}$ y **B** una matriz de tamaño $\boldsymbol{n} \times k$. El **producto** $\mathbf{C} = \mathbf{AB}$ es una matriz de tamaño $m \times k$, donde cada componente c_{ij} se calcula como el **producto punto** de la **fila** i **de A** con la **columna** j **de B**.

$$c_{ij} = \sum_{l=1}^{n} a_{il} \cdot b_{lj}$$

Regla de las Dimensiones Internas

Para que el producto **AB** sea posible, el número de **columnas de A** debe ser igual al número de **filas de B**.

$$\underbrace{\mathbf{A}}_{m \times \boldsymbol{n}} \cdot \underbrace{\mathbf{B}}_{\boldsymbol{n} \times k} = \underbrace{\mathbf{C}}_{m \times k}$$

Las dimensiones "internas" $(\boldsymbol{n})$ deben coincidir y "desaparecen", mientras que las dimensiones "externas" (m, k) definen el tamaño del resultado.

Tema: Multiplicación de Matrices: Visualización del Proceso

Para calcular el elemento c_{ij} del producto $\mathbf{C} = \mathbf{AB}$:

$$\begin{pmatrix} \cdots & \cdots & \cdots & \\ a_{i1} & a_{i2} & \cdots & a_{in} \\ \cdots & \cdots & \cdots & \end{pmatrix} \begin{pmatrix} \cdots & b_{1j} & \cdots \\ \cdots & b_{2j} & \cdots \\ \vdots & \vdots & \vdots \\ \cdots & b_{nj} & \cdots \end{pmatrix} = \begin{pmatrix} c_{11} & \cdots & c_{1k} \\ \vdots & c_{ij} & \vdots \\ c_{m1} & \cdots & c_{mk} \end{pmatrix}$$

$$c_{ij} = (a_{i1} \cdot b_{1j}) + (a_{i2} \cdot b_{2j}) + \cdots + (a_{in} \cdot b_{nj})$$

Tema: Multiplicación de Matrices: Propiedad Clave: NO es Conmutativa

¡Advertencia Crucial!

A diferencia de la multiplicación de números, la multiplicación de matrices **en general no es conmutativa**.

$$\mathbf{AB} \neq \mathbf{BA}$$

Puede ocurrir que:

1. **AB** esté definido, pero **BA** no.
2. Ambos estén definidos, pero resulten en matrices de **diferente tamaño**.
3. Ambos estén definidos, sean del mismo tamaño, pero sus **valores sean distintos**.

Observación (Producto de Hadamard)

La multiplicación elemento a elemento, $(C)_{ij} = a_{ij} \cdot b_{ij}$, sí existe y se llama **producto de Hadamard** (o producto Schur), denotado $\mathbf{A} \odot \mathbf{B}$. Es común en librerías como NumPy (usando el operador '*'), pero **no es la multiplicación matricial estándar** del álgebra lineal (que en NumPy es '@' o '.dot()').

Ejemplo

Sean $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 3 & 2 & 1 \end{pmatrix} \in \mathbb{R}^{2\times 3}$ y $\mathbf{B} = \begin{pmatrix} 0 & 2 \\ 1 & -1 \\ 0 & 1 \end{pmatrix} \in \mathbb{R}^{3\times 2}$.

Calculemos $\mathbf{C} = \mathbf{AB}$. El resultado será una matriz de 2×2.

$$\mathbf{AB} = \begin{pmatrix} \mathbf{1} & \mathbf{2} & \mathbf{3} \\ 3 & 2 & 1 \end{pmatrix} \begin{pmatrix} \mathbf{0} & 2 \\ \mathbf{1} & -1 \\ \mathbf{0} & 1 \end{pmatrix} = \begin{pmatrix} (1\cdot 0 + 2\cdot 1 + 3\cdot 0) & (1\cdot 2 + 2\cdot(-1) + 3\cdot 1) \\ (3\cdot 0 + 2\cdot 1 + 1\cdot 0) & (3\cdot 2 + 2\cdot(-1) + 1\cdot 1) \end{pmatrix}$$

$$\mathbf{AB} = \begin{pmatrix} 2 & 3 \\ 2 & 5 \end{pmatrix} \in \mathbb{R}^{2\times 2}$$

Ejemplo

Usando las mismas matrices, ahora calculemos $\mathbf{D} = \mathbf{BA}$. El resultado será una matriz de 3×3.

$$\mathbf{BA} = \begin{pmatrix} 0 & 2 \\ 1 & -1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 3 \\ 3 & 2 & 1 \end{pmatrix} = \begin{pmatrix} (0 \cdot 1 + 2 \cdot 3) & (0 \cdot 2 + 2 \cdot 2) & (0 \cdot 3 + 2 \cdot 1) \\ (1 \cdot 1 + (-1) \cdot 3) & (1 \cdot 2 + (-1) \cdot 2) & (1 \cdot 3 + (-1) \cdot 1) \\ (0 \cdot 1 + 1 \cdot 3) & (0 \cdot 2 + 1 \cdot 2) & (0 \cdot 3 + 1 \cdot 1) \end{pmatrix}$$

$$\mathbf{BA} = \begin{pmatrix} 6 & 4 & 2 \\ -2 & 0 & 2 \\ 3 & 2 & 1 \end{pmatrix} \in \mathbb{R}^{3 \times 3}$$

Claramente, $\mathbf{AB} \neq \mathbf{BA}$.

Definición (Matriz Identidad)

La **matriz identidad** de tamaño $n \times n$, denotada $\mathbf{I}_n$, es una matriz cuadrada con unos en la diagonal principal y ceros en el resto de las posiciones.

$$\mathbf{I}_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{I}_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \ldots$$

La matriz identidad es el **elemento neutro** de la multiplicación de matrices, análogo al número 1 en la multiplicación de escalares.

Proposición (Propiedad de la Identidad)

Para cualquier matriz $\mathbf{A}$ de tamaño $m \times n$:

$$\mathbf{I}_m\mathbf{A} = \mathbf{A} \quad \text{y} \quad \mathbf{A}\mathbf{I}_n = \mathbf{A}$$

Definición (Matriz Inversa)

Sea $\mathbf{A}$ una matriz **cuadrada** de tamaño $n \times n$. Si existe una matriz $\mathbf{A}^{-1}$ de tamaño $n \times n$ tal que:

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n$$

entonces $\mathbf{A}^{-1}$ es la **matriz inversa** de $\mathbf{A}$.

Existencia de la Inversa

- Solo las matrices **cuadradas** pueden tener inversa.
- No toda matriz cuadrada tiene inversa.
- Si $\mathbf{A}^{-1}$ existe, $\mathbf{A}$ se llama **invertible** o **no singular**.
- Si no existe, $\mathbf{A}$ se llama **no invertible** o **singular**.

Proposición (Inversa de una Matriz 2x2)

Para una matriz general de 2×2, $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, su inversa existe si y solo si su **determinante** no es cero.
El determinante es $\det(\mathbf{A}) = ad - bc$.
Si $\det(\mathbf{A}) \neq 0$, la inversa es:

$$\mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

Ejemplo

Sea $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 4 & 2 \end{pmatrix}$.

- $\det(\mathbf{A}) = (3)(2) - (1)(4) = 6 - 4 = 2$.
- Como $2 \neq 0$, la inversa existe.
- $\mathbf{A}^{-1} = \frac{1}{2} \begin{pmatrix} 2 & -1 \\ -4 & 3 \end{pmatrix} = \begin{pmatrix} 1 & -0{,}5 \\ -2 & 1{,}5 \end{pmatrix}$.

Definiremos el determinante de manera mas formal y para matrices mas grandes en las próximas clases.

Tema: Otra Operación Clave: Matriz Transpuesta

Definición (Matriz Transpuesta)

Sea $\mathbf{A}$ una matriz de tamaño $m \times n$. Su **transpuesta**, denotada $\mathbf{A}^\top$, es una matriz de tamaño $n \times m$ que se obtiene al intercambiar las filas por las columnas.

$$(\mathbf{A}^\top)_{ij} = a_{ji}$$

Ejemplo

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \quad \Longrightarrow \quad \mathbf{A}^\top = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

La primera fila de $\mathbf{A}$ se convierte en la primera columna de $\mathbf{A}^\top$, y así sucesivamente.

Definición (Matriz Simétrica)

Una matriz **cuadrada A** es **simétrica** si es igual a su propia transpuesta.

$$\mathbf{A} = \mathbf{A}^\top$$

Esto implica que $a_{ij} = a_{ji}$ para todo i, j. La matriz es un .espejo.a través de su diagonal principal.

Ejemplo

Las siguientes matrices son simétricas:

$$\begin{pmatrix} 2 & 5 \\ 5 & 7 \end{pmatrix}, \quad \begin{pmatrix} 1 & -2 & 3 \\ -2 & 4 & 0 \\ 3 & 0 & 5 \end{pmatrix}$$

Observación

Las matrices de covarianza y correlación, fundamentales en estadística, son ejemplos de matrices simétricas.

Tema: Propiedades: Resumen General (1/2)

Asumiendo que las dimensiones de las matrices son compatibles para cada operación:

Propiedades de Suma y Escalar:

- $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$ (Conmutatividad de la suma)
- $(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C})$ (Asociatividad de la suma)
- $\lambda(\mathbf{A} + \mathbf{B}) = \lambda\mathbf{A} + \lambda\mathbf{B}$ (Distributividad)

Propiedades de la Multiplicación:

- $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$ (Asociatividad de la multiplicación)
- $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$ (Distributividad)
- $\mathbf{IA} = \mathbf{AI} = \mathbf{A}$ (Elemento neutro)
- $\mathbf{AB} \neq \mathbf{BA}$ (NO conmutatividad)

Tema: Propiedades: Resumen General (2/2)

Propiedades de la Transpuesta:

- $(\mathbf{A}^\top)^\top = \mathbf{A}$
- $(\mathbf{A} + \mathbf{B})^\top = \mathbf{A}^\top + \mathbf{B}^\top$
- $(\lambda \mathbf{A})^\top = \lambda \mathbf{A}^\top$
- $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$ (Invierte el orden)

Propiedades de la Inversa:

- $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$
- $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ (Invierte el orden)
- $(\mathbf{A}^\top)^{-1} = (\mathbf{A}^{-1})^\top$

Actividad (Ranking de Jugadores)

En un torneo de tenis, n jugadores compiten entre sí. Los resultados se guardan en una matriz $\mathbf{R}$ de $n \times n$:

$$R_{ij} = \begin{cases} 1 & \text{si el jugador } i \text{ vence al jugador } j, \\ 0 & \text{en caso contrario (incluyendo } i = j). \end{cases}$$

A cada jugador se le asigna un puntaje S_i basado en sus victorias directas y en las victorias de la gente a la que venció:

$$S_i = \underbrace{\sum_{j=1}^{n} R_{ij}}_{\text{Victorias directas}} + \frac{1}{2} \underbrace{\sum_{j=1}^{n} (R^2)_{ij}}_{\text{Victorias indirectas}}$$

Observación

La entrada $(R^2)_{ij}$ cuenta el número de "caminos de largo 2" del jugador i al j. Es decir, el número de jugadores k tales que i venció a k y k venció a j. Sumar sobre j nos da el total de victorias de la gente a la que i ha ganado.

Actividad (Tarea)

Dado el siguiente torneo con cuatro jugadores:

$$\mathbf{R} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

donde las filas/columnas corresponden a los jugadores 1, 2, 3 y 4.

Pasos a seguir:

1. **Calcular $\mathbf{R}^2$**: Multiplique la matriz $\mathbf{R}$ por sí misma.
2. **Calcular los puntajes S_i**: Para cada jugador i (cada fila), aplique la fórmula de puntuación. Puede expresar esto de forma vectorial.
3. **Clasificar a los jugadores**: Ordene los jugadores de mayor a menor puntaje e interprete el resultado. ¿Por qué este sistema de ranking podría ser más justo que solo contar las victorias directas?

Actividad (Escenario)

Un inversor desea comparar la evolución de un capital inicial C invertido en 6 compañías (Apple, Amazon, etc.) desde el 01/01/2024 hasta el 18/02/2025.

Datos de entrada:

- $\mathbf{P} \in \mathbb{R}^{T \times 6}$: Matriz de precios diarios, con T días y 6 compañías.
- Vector de precios del S&P500 para el mismo período.
- Capital inicial C (ej. valor del S&P500 el primer día).

Estrategias a comparar:

1. **Buy and Hold:** Invertir $C/6$ en cada acción el primer día y no volver a operar.
2. **Rebalanceo Diario:** Reajustar la cartera cada día para que el valor invertido en cada acción sea siempre el $1/6$ del total.

Objetivo: Usando operaciones de álgebra lineal, calcular y graficar la riqueza acumulada diaria de ambas estrategias y compararla con el S&P500.

Tarea de Implementación

Tu misión es:

1. Obtener los datos de precios diarios para las 6 acciones y el S&P500 (usando librerías como ‘yfinance‘).
2. Implementar las dos estrategias de cálculo de riqueza usando NumPy, aprovechando las operaciones matriciales y vectoriales.
3. Generar una gráfica para comparar visualmente el rendimiento.

Tema: Inversión: Estrategia 1 (Buy and Hold)

Lógica: La cantidad de acciones de cada compañía se fija el primer día y permanece constante.
Pasos con Álgebra Lineal:

1. Sea $\mathbf{p}_1^\top$ el primer vector fila de $\mathbf{P}$ (precios del día 1).
2. El capital asignado a cada acción es $C/6$.
3. El vector de **número de acciones** $\mathbf{q} \in \mathbb{R}^{6\times 1}$ se calcula elemento a elemento:

$$q_i = \frac{C/6}{p_{1,i}} \implies \mathbf{q} = \frac{C}{6}\left(\frac{1}{\mathbf{p}_1}\right)^\top$$

   (donde la división es elemento a elemento).
4. El valor de la cartera para cada día t es el producto punto de los precios de ese día, $\mathbf{p}_t^\top$, con el vector de acciones $\mathbf{q}$.
5. El vector de **riqueza acumulada** $\mathbf{W}^{(1)} \in \mathbb{R}^{T\times 1}$ se calcula de forma compacta con una única multiplicación matriz-vector:

$$\mathbf{W}^{(1)} = \mathbf{P}\mathbf{q}$$

Tema: Inversión: Estrategia 2 (Rebalanceo Diario)

Lógica: La *proporción* de la riqueza en cada acción es constante, no la cantidad de acciones.
Pasos con Álgebra Lineal:

1. El vector de **pesos** (weights) es constante: $\mathbf{w} = \frac{1}{6}\left(1,1,1,1,1,1\right)^\top$.
2. Se necesita la matriz de **retornos simples diarios** $\mathbf{R} \in \mathbb{R}^{(T-1)\times 6}$, donde $R_{t,i} = \frac{P_{t+1,i} - P_{t,i}}{P_{t,i}}$.
3. El retorno de la cartera en el día t es el producto punto del vector de pesos con el vector de retornos de ese día ($\mathbf{r}_t^\top$, la fila t de $\mathbf{R}$):

$$r_t^{(p)} = \mathbf{w}^\top \mathbf{r}_t$$

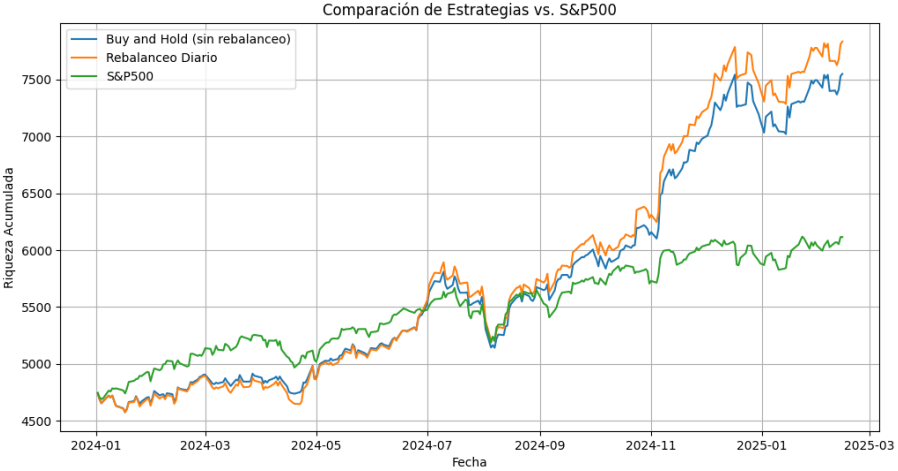
4. La riqueza acumulada $\mathbf{W}^{(2)}$ se calcula de forma iterativa:

$$W_1^{(2)} = C, \quad W_{t+1}^{(2)} = W_t^{(2)} \cdot (1 + r_t^{(p)})$$

   Esto se puede expresar como un producto acumulado. Aunque no es una sola multiplicación matricial, cada paso se basa en un producto punto.

Tema: Inversión: Resultados

Al implementar ambas estrategias en Python y graficar los resultados junto con el S&P500, se obtiene una visualización como la siguiente:



Operaciones Algebraicas con Tensores

Tema: Operaciones con Tensores: Introducción

Si los tensores son los contenedores de datos de la ciencia de datos moderna, las operaciones tensoriales son el motor que impulsa los algoritmos. Estas operaciones generalizan lo que hemos visto para matrices y nos permiten realizar cálculos complejos de forma estructurada y eficiente.

En esta sección, exploraremos las operaciones fundamentales que se realizan en frameworks como **TensorFlow** y **PyTorch**:

- Operaciones elemento a elemento.
- El poderoso concepto de *Broadcasting*.
- Diferentes formas de "multiplicar" tensores: Hadamard, Contracción y Producto en Modo-k.
- Manipulación de la estructura del tensor: Transposición y Remodelado.

Tema: Operaciones Elemento a Elemento: Definición

Estas son las operaciones más sencillas y directas. Funcionan de manera análoga a la suma de matrices.

Definición (Operaciones Elemento a Elemento)

Dados dos tensores $\boldsymbol{\mathcal{A}}$ y $\boldsymbol{\mathcal{B}}$ que tienen **exactamente la misma forma (shape)**, su suma, resta o producto de Hadamard (elemento a elemento) se definen componente a componente:

- **Suma/Resta:** $(\boldsymbol{\mathcal{A}} \pm \boldsymbol{\mathcal{B}})_{i_1,\ldots,i_N} = \mathcal{A}_{i_1,\ldots,i_N} \pm \mathcal{B}_{i_1,\ldots,i_N}$
- **Producto Hadamard:** $(\boldsymbol{\mathcal{A}} \odot \boldsymbol{\mathcal{B}})_{i_1,\ldots,i_N} = \mathcal{A}_{i_1,\ldots,i_N} \cdot \mathcal{B}_{i_1,\ldots,i_N}$

Multiplicación por un Escalar (λ):
Cada elemento del tensor se multiplica por el escalar.

$$(\lambda \boldsymbol{\mathcal{A}})_{i_1,\ldots,i_N} = \lambda \cdot \mathcal{A}_{i_1,\ldots,i_N}$$

Tema: Operaciones Flexibles: Broadcasting

¿Qué pasa si los tensores no tienen la misma forma? Las librerías modernas utilizan una poderosa regla llamada **Broadcasting** para inferir cómo realizar la operación.

Idea del Broadcasting

Si un tensor tiene una dimensión de tamaño 1 (o le falta una dimensión), *Broadcasting* "estira" o "duplica" virtualmente ese tensor a lo largo de esa dimensión para que su forma coincida con la del otro tensor.

Ejemplo (Sumar un vector a una matriz)

Sea $\boldsymbol{M} \in \mathbb{R}^{3\times 2}$ y $\boldsymbol{v} \in \mathbb{R}^{1\times 2}$.

$$\boldsymbol{M} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}, \quad \boldsymbol{v} = \begin{pmatrix} 10 & 20 \end{pmatrix}$$

Para sumar $\boldsymbol{M} + \boldsymbol{v}$, *broadcasting* "estira" $\boldsymbol{v}$ para que coincida con la forma de $\boldsymbol{M}$:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} + \begin{pmatrix} 10 & 20 \\ 10 & 20 \\ 10 & 20 \end{pmatrix}_{\text{virtual}} = \begin{pmatrix} 11 & 22 \\ 13 & 24 \\ 15 & 26 \end{pmatrix}$$

La **contracción de tensores** es la generalización de la multiplicación de matrices y el producto punto. La idea es multiplicar los tensores y luego sumar a lo largo de los índices "compartidos".

Definición (Contracción)
Es una operación que reduce el orden de un par de tensores al sumar sobre un conjunto de índices especificados.

Notación Einstein (einsum)

Una forma muy poderosa y común de expresar contracciones es con la notación 'einsum'. Se especifican los índices de entrada y los índices deseados de salida. Los índices que aparecen en la entrada pero no en la salida se suman (contraen).
Multiplicación de matrices: '$ij, jk \to ik$'
Producto punto: '$i, i \to$'
Transpuesta de matriz: '$ij \to ji$'

Ejemplo (Contracción Tensor-Matriz)
Sea $\mathcal{T} \in \mathbb{R}^{I \times J \times K}$ y $\boldsymbol{M} \in \mathbb{R}^{K \times L}$. Podemos contraer el último índice de $\mathcal{T}$ con el primero de $\boldsymbol{M}$.

$$\mathcal{C}_{ijk} = \sum_{l} \mathcal{T}_{ijl} \boldsymbol{M}_{lk} \quad (\text{einsum: } 'ijl, lk \to ijk')$$

El resultado $\mathcal{C}$ es un tensor de orden 3 con forma (I, J, K).

Un caso especial y muy útil de contracción es el **Producto en Modo-k**, donde una matriz opera sobre un único eje (o "modo") de un tensor.

Definición (Producto en Modo-k)

Es la multiplicación de un tensor $\boldsymbol{\mathcal{T}} \in \mathbb{R}^{d_1 \times \cdots \times d_k \times \cdots \times d_N}$ por una matriz $\boldsymbol{M} \in \mathbb{R}^{J \times d_k}$ a lo largo del modo k. La operación transforma el eje k de tamaño d_k a un nuevo tamaño J.

$$(\boldsymbol{\mathcal{T}} \times_k \boldsymbol{M})_{i_1 \ldots j \ldots i_N} = \sum_{i_k=1}^{d_k} \mathcal{T}_{i_1 \ldots i_k \ldots i_N} \cdot M_{j,i_k}$$

El tensor resultante tiene forma $(d_1, \ldots, J, \ldots, d_N)$.

Observación

Piénselo como aplicar una **transformación lineal** (la matriz $\boldsymbol{M}$) a todos los vectores que se forman al "rebanar" el tensor a lo largo de su k-ésimo eje.

Transposición (Permutación de Ejes)
Generaliza la transpuesta de una matriz. No se limita a intercambiar dos ejes, sino que permite **reordenar los ejes** en cualquier forma.

Ejemplo

Un lote (batch) de imágenes se almacena comúnmente en un tensor de forma '(B, H, W, C)': (Batch, Altura, Ancho, Canales).
Si una librería de visualización espera la forma '(B, C, H, W)', podemos usar una transposición (permutación):

$$\mathcal{T}_{\text{nuevo}} = \mathcal{T}_{\text{original}}.\text{permute}(0, 3, 1, 2)$$

El nuevo tensor tiene los mismos datos, pero los ejes están en un orden diferente.

Remodelado (Reshape)
Cambia la forma de un tensor sin cambiar el número total de elementos ni su orden en memoria.

Ejemplo

Un tensor de forma $(2, 6)$ puede ser remodelado a $(4, 3)$, $(12, 1)$, o incluso a $(2, 2, 3)$.

Actividad (Modelo de Predicción de Ventas)

Tenemos datos de ventas en un tensor $\mathcal{T} \in \mathbb{R}^{3\times4\times2}$, donde las dimensiones representan:

- **Modo 1 (tamaño 3):** Tres productos distintos (P1, P2, P3).
- **Modo 2 (tamaño 4):** Cuatro regiones geográficas (R1, R2, R3, R4).
- **Modo 3 (tamaño 2):** Dos períodos de tiempo (T1, T2).

Queremos aplicar una transformación que combine la información de los 3 productos en 2 "factores de producto"(ej. "gama alta", "gama baja"). Para ello, usamos una matriz de transformación $\boldsymbol{M} \in \mathbb{R}^{2\times3}$.

$$\boldsymbol{M} = \begin{pmatrix} 1{,}2 & 0{,}8 & 1{,}0 \\ 0{,}9 & 1{,}1 & 1{,}3 \end{pmatrix}$$

Esta operación es un **Producto en Modo-1**.

Tema: Aplicación: Modelo de Ventas (Cálculo)

La operación a realizar es $\mathcal{T}' = \mathcal{T} \times_1 \boldsymbol{M}$. El resultado $\mathcal{T}'$ tendrá forma $(2 \times 4 \times 2)$.
La fórmula para cada elemento es:

$$\mathcal{T}'_{j,i_2,i_3} = \sum_{i_1=1}^{3} \mathcal{T}_{i_1,i_2,i_3} \cdot M_{j,i_1}$$

Ejemplo (Cálculo Manual para Región 1, Tiempo 1)

Supongamos que los datos para $(i_2 = 1, i_3 = 1)$ son el vector $\begin{pmatrix} 10 \\ 8 \\ 5 \end{pmatrix}$ (ventas de P1, P2, P3).

La nueva primera componente $(j = 1)$ será:

$$\mathcal{T}'_{1,1,1} = (1{,}2 \cdot 10) + (0{,}8 \cdot 8) + (1{,}0 \cdot 5) = 12 + 6{,}4 + 5 = 23{,}4$$

La nueva segunda componente $(j = 2)$ será:

$$\mathcal{T}'_{2,1,1} = (0{,}9 \cdot 10) + (1{,}1 \cdot 8) + (1{,}3 \cdot 5) = 9 + 8{,}8 + 6{,}5 = 24{,}3$$

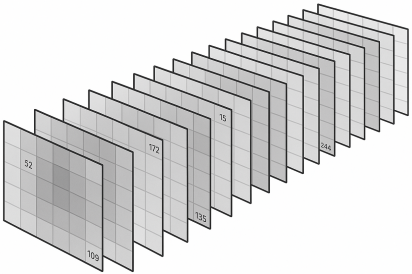
El vector de datos original $(10, 8, 5)^\top$ se ha transformado en $(23{,}4, 24{,}3)^\top$ por la matriz $\boldsymbol{M}$. Este proceso se repite para todas las combinaciones de región y tiempo.

Actividad (Análisis de un Vídeo)

Un vídeo digital es una secuencia de imágenes. Podemos representarlo como un tensor de **orden 4**.

Tarea 1: Carga y Representación

- Consiga un vídeo corto (aprox. 5 segundos) y cárguelo en Python (usando librerías como 'OpenCV' o 'imageio').
- Represente el vídeo como un tensor $\mathcal{V}$ con forma (F, H, W, C), donde:
  - F: número de fotogramas (frames).
  - H: altura de la imagen.
  - W: ancho de la imagen.
  - C: canales de color (usualmente 3 para RGB).



Actividad (Manipulación del Tensor de Vídeo)

Tarea 2: Fotograma Promedio (Reducción)

Calcule y muestre el fotograma promedio del vídeo. Esto implica realizar una operación de **promedio (mean) a lo largo del eje 0 (eje de fotogramas)**. ¿Qué efecto visual se observa?

Tarea 3: Ajuste de Brillo (Multiplicación por Escalar)

Multiplique el tensor de vídeo completo por un escalar $\lambda > 1$ (ej. $\lambda = 1{,}5$) y por un escalar $0 < \lambda < 1$ (ej. $\lambda = 0{,}7$). Observe cómo cambia el brillo del vídeo.

Tarea 4: Filtro de Color (Contracción)

Para cada píxel en cada fotograma, el color es un vector de 3 componentes (R, G, B). Aplique un filtro multiplicando cada uno de estos vectores por la matriz:

$$\boldsymbol{F}_S = \begin{pmatrix} 0{,}393 & 0{,}769 & 0{,}189 \\ 0{,}349 & 0{,}686 & 0{,}168 \\ 0{,}272 & 0{,}534 & 0{,}130 \end{pmatrix}$$

Esta operación es un **producto en el modo 3** (o modo de canal). En ‘einsum‘, se escribiría como ‘‘$fhwc, dc \to fhwd$‘‘, donde ‘d‘ es la nueva dimensión de canal. ¿Cuál es el resultado visual?

Conclusión: Resumen y Próximos Pasos

Hoy hemos construido el lenguaje fundamental del álgebra para la ciencia de datos. Hemos pasado de representar datos como simples números a estructurarlos en **vectores**, **matrices** y **tensores**.

Conceptos Clave Vistos:

- Datos como objetos algebraicos.
- Operaciones matriciales: suma, producto escalar y el crucial **producto matricial**.
- El poder de los tensores y sus operaciones: *broadcasting*, contracción y permutación.

El Puente a la Práctica:

- Vimos cómo estos conceptos modelan desde **redes sociales** y **finanzas** hasta el **análisis de texto e imágenes**.
- Las actividades prácticas (ranking, portafolios, vídeo) demuestran que estas no son solo ideas abstractas.

¿Qué sigue?

En el futuro inmediato, usaremos este lenguaje para explorar conceptos más profundos como la **geometría de los datos**. Analizaremos cómo medir distancias y similitudes usando **normas** y el **producto interno**. ¡Prepárense para ver los datos desde una nueva perspectiva!

Distancias y Métricas Cuantificando la diferencia entre observaciones

Distancias y Métricas: Introducción: Motivación

En la vida diaria, la "distancia" es una idea intuitiva. En ciencia de datos, debemos formalizarla. La forma en que medimos la distancia (o similitud) entre puntos de datos es una de las decisiones más críticas que tomamos.

La elección de una métrica de distancia impacta directamente en algoritmos como:

- **Clasificación:** k-Vecinos Más Cercanos (k-NN) depende de quiénes son los "vecinos".
- **Agrupamiento (Clustering):** Algoritmos como K-Means o DBSCAN agrupan puntos basándose en su proximidad.
- **Sistemas de Recomendación:** Encontrar usuarios con gustos "similares".
- **Detección de Anomalías:** Identificar puntos que están "lejos" de todos los demás.

Idea Central

No existe una "distancia universal". La métrica correcta depende del **tipo de datos** (numéricos, categóricos, texto) y del **contexto del problema**.

Definición (Métrica o Función de Distancia)

Una función $d : V \times V \to \mathbb{R}$ es una **métrica** en un conjunto V si para cualquier $\mathbf{x}, \mathbf{y}, \mathbf{z} \in V$, cumple las siguientes cuatro propiedades:

1. **No negatividad:** $d(\mathbf{x}, \mathbf{y}) \geq 0$. (La distancia no puede ser negativa).
2. **Identidad de los indiscernibles:** $d(\mathbf{x}, \mathbf{y}) = 0 \iff \mathbf{x} = \mathbf{y}$. (La distancia es cero solo si los puntos son idénticos).
3. **Simetría:** $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$. (La distancia de A a B es la misma que de B a A).
4. **Desigualdad del Triángulo:** $d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z})$. (El camino directo es siempre el más corto).

Un conjunto V junto con una métrica d se llama un **espacio métrico**.

Métricas para Datos Numéricos: La Familia Minkowski: Definición General

Para datos numéricos en un espacio vectorial $\mathbb{R}^n$, una familia muy versátil de distancias es la **distancia de Minkowski**.

Definición

Sean $\mathbf{x} = (x_1, \ldots, x_n)$ y $\mathbf{y} = (y_1, \ldots, y_n)$ dos vectores en $\mathbb{R}^n$. Para un parámetro $p \geq 1$, la distancia de Minkowski de orden p se define como:

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}}$$

Esta función es también conocida como la **norma** ℓ_p de la diferencia entre los vectores, $\|\mathbf{x} - \mathbf{y}\|_p$.

Observación

Cuando $p \geq 1$, la distancia de Minkowski satisface todas las propiedades de una métrica. El parámetro p nos permite ajustar la "sensibilidad" de la distancia a las grandes diferencias en coordenadas individuales.

Métricas para Datos Numéricos: La Familia Minkowski: Casos Especiales

Ajustando el parámetro p, obtenemos las distancias más comunes usadas en ciencia de datos.

Distancia de Manhattan ($p = 1$) *(También norma ℓ_1 o "City Block")*

$$d_1(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} |x_i - y_i|$$

Suma las diferencias absolutas a lo largo de cada eje. Es como moverse por las calles de una ciudad cuadriculada.

Distancia Euclidiana ($p = 2$) *(También norma ℓ_2)*

$$d_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Es la distancia "en línea recta" que todos conocemos. Es la métrica por defecto en muchos algoritmos.

Distancia de Chebyshev ($p \to \infty$)

En el límite cuando p tiende a infinito, la distancia de Minkowski se convierte en la **Distancia de Chebyshev** (o norma ℓ_∞):

$$d_\infty(\mathbf{x}, \mathbf{y}) = \max_i |x_i - y_i|$$

Esta métrica considera únicamente la **máxima diferencia** encontrada en cualquiera de las coordenadas.

Métricas para Datos Numéricos: Comparación Visual

¿Cómo se ven los "círculos" (conjunto de puntos a la misma distancia de un centro) para cada métrica?

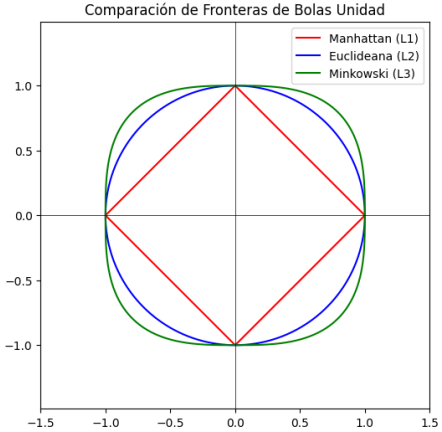


Figura: Círculos unitarios para $p = 1$ (rombo), $p = 2$ (círculo), y $p \to \infty$ (cuadrado).

Interpretación:

- **Manhattan ($p = 1$):** Valora los caminos a lo largo de los ejes.
- **Euclidiana ($p = 2$):** Valora los caminos "diagonales" o directos.
- **Chebyshev ($p = \infty$):** Solo se preocupa por la dimensión con la mayor discrepancia.

Métricas para Datos Numéricos: Ejemplo de Cálculo

Ejemplo (Preferencias de Usuarios)

Comparemos las preferencias de dos usuarios sobre tres géneros de películas (acción, comedia, drama), en una escala de 1 a 5.
$\mathbf{u} = (4, 2, 5)$ y $\mathbf{v} = (3, 5, 1)$. La diferencia es $\mathbf{u} - \mathbf{v} = (1, -3, 4)$.

- **Distancia de Manhattan ($p = 1$):** $d_1(\mathbf{u}, \mathbf{v}) = |1| + |-3| + |4| = 1 + 3 + 4 = \mathbf{8}$
- **Distancia Euclidiana ($p = 2$):** $d_2(\mathbf{u}, \mathbf{v}) = \sqrt{1^2 + (-3)^2 + 4^2} = \sqrt{1 + 9 + 16} = \sqrt{26} \approx \mathbf{5{,}099}$
- **Distancia de Minkowski ($p = 3$):**
  $d_3(\mathbf{u}, \mathbf{v}) = \left(1^3 + |-3|^3 + 4^3\right)^{1/3} = (1 + 27 + 64)^{1/3} = \sqrt[3]{92} \approx \mathbf{4{,}514}$
- **Distancia de Chebyshev ($p = \infty$):** $d_\infty(\mathbf{u}, \mathbf{v}) = \text{máx}(|1|, |-3|, |4|) = \mathbf{4}$

Observación

Note que a medida que p aumenta, el resultado se acerca al valor máximo de las diferencias individuales. La distancia euclidiana penaliza más las grandes diferencias (como el -3 y el 4) que la de Manhattan.

Métricas para Datos Numéricos: Sensibilidad a Outliers y Escala

Sensibilidad a Outliers:

- La **Distancia Euclidiana ($p = 2$)** eleva las diferencias al cuadrado, por lo que una única gran diferencia (un outlier) puede **dominar** el cálculo total de la distancia.
- La **Distancia de Manhattan ($p = 1$)** es más **robusta** a outliers porque las diferencias no se elevan al cuadrado.

Sensibilidad a la Escala de las Variables:

- Todas las distancias de Minkowski son sensibles a la escala. Si una variable está en una escala mucho mayor que las otras (ej. ingresos vs. número de hijos), dominará el cálculo de la distancia.
- **Solución:** ¡Estandarizar los datos! Llevar todas las variables a una escala comparable (ej. media 0 y desviación estándar 1) es casi siempre un paso de preprocesamiento necesario antes de usar estas métricas.

Actividad

Ejercicio 1: Comparación de Distancias Dados $\mathbf{x} = (1, 5, 3, 7)$ y $\mathbf{y} = (4, 2, 8, 1)$ en $\mathbb{R}^4$. Calcule $d_1(\mathbf{x}, \mathbf{y})$, $d_2(\mathbf{x}, \mathbf{y})$, y $d_\infty(\mathbf{x}, \mathbf{y})$. Analice cómo varían los valores y por qué.

Ejercicio 2: k-NN y Elección de Métrica Un clasificador k-NN (con $k = 3$) tiene los siguientes puntos etiquetados:

$$(1, 2) \to \text{Clase A}, \quad (2, 3) \to \text{Clase B}, \quad (4, 1) \to \text{Clase A}, \quad (6, 2) \to \text{Clase B}$$

Clasifique el nuevo punto $\mathbf{z} = (3, 2)$ usando:

1. Distancia Euclidiana para encontrar los 3 vecinos más cercanos.
2. Distancia de Manhattan para encontrar los 3 vecinos más cercanos.
3. ¿Cambia la clasificación? Discuta qué métrica podría ser más apropiada si sospecha que los datos tienen mucho ruido en una dirección.

Más Allá de los Números: Introducción: Métricas para Otros Tipos de Datos

No todos los datos son vectores de números reales. En ciencia de datos, trabajamos constantemente con:

- **Datos Categóricos:** Vectores binarios (presencia/ausencia de un rasgo), etiquetas, etc.
- **Datos Textuales:** Cadenas de caracteres, como tweets, descripciones de productos o secuencias de ADN.
- **Datos de Conjuntos:** Listas de intereses de un usuario, palabras en un documento (bag-of-words).

Para estos casos, necesitamos métricas especializadas que capturen la noción de "diferencia" de manera apropiada. Exploraremos tres de las más importantes:

1. **Distancia de Hamming:** Para secuencias de longitud fija.
2. **Distancia de Jaccard:** Para comparar conjuntos.
3. **Distancia de Levenshtein:** Para secuencias de longitud variable (texto).

Métricas Categóricas: Distancia de Hamming

Definición

La **Distancia de Hamming** se aplica a dos secuencias (o vectores) $\mathbf{x}$ y $\mathbf{y}$ de la **misma longitud** n. Se define como el **número de posiciones en las que los símbolos correspondientes son diferentes**.

$$d_H(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} \mathbb{I}(x_i \neq y_i)$$

donde $\mathbb{I}(\cdot)$ es la función indicadora (1 si la condición es verdadera, 0 si es falsa).

Ejemplo (Vectores Binarios)

Sean $\mathbf{x} = (1, 0, 1, 1, 0)$ y $\mathbf{y} = (1, 1, 1, 0, 0)$.

Posición:	1	2	3	4	5
$\mathbf{x}$:	1	0	1	1	0
$\mathbf{y}$:	1	1	1	0	0
Difieren?:	No	**Sí**	No	**Sí**	No

Difieren en las posiciones 2 y 4. Por lo tanto, $d_H(\mathbf{x}, \mathbf{y}) = 2$.

Ejemplo (Cadenas de Texto)

$d_H(\text{“gato”}, \text{“pato”}) = 1.$ $\quad d_H(\text{“sol”}, \text{“sal”}) = 1.$

Observación

La distancia de Hamming es una métrica válida y es muy útil en bioinformática (comparar secuencias de ADN) y teoría de la codificación (detectar errores en la transmisión de bits).

Métricas de Conjuntos: Distancia de Jaccard

Cuando nuestros datos son conjuntos de elementos (como las palabras en un documento o los productos que un cliente ha comprado), necesitamos una métrica que compare estos conjuntos.

Definición (Índice de Jaccard (Similitud))

Para dos conjuntos no vacíos A y B, el **Índice de Jaccard** mide su similitud como la proporción entre el tamaño de su intersección y el tamaño de su unión.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Definición (Distancia de Jaccard)

La **Distancia de Jaccard** se define como 1 menos el índice de similitud.

$$d_J(A, B) = 1 - J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cup B| - |A \cap B|}{|A \cup B|} = \frac{|A \Delta B|}{|A \cup B|}$$

Donde $A \Delta B$ es la diferencia simétrica.

Ejemplo

$D_1 = \{\text{python, data, nlp}\}, \quad D_2 = \{\text{data, java, stats, python}\}$

- $A \cap B = \{\text{python, data}\} \implies |A \cap B| = 2.$
- $A \cup B = \{\text{python, data, nlp, java, stats}\} \implies |A \cup B| = 5.$
- $d_J(A, B) = 1 - \frac{2}{5} = \frac{3}{5} = 0{,}6.$

Métricas Textuales: Distancia de Levenshtein

¿Qué pasa si las secuencias que queremos comparar no tienen la misma longitud? Hamming no nos sirve. Aquí entra la distancia de edición.

Definición (Distancia de Levenshtein (o de Edición))

La **Distancia de Levenshtein** entre dos cadenas de texto es el **número mínimo de ediciones de un solo carácter** (inserciones, eliminaciones o sustituciones) necesarias para cambiar una cadena por la otra.

Ejemplo

Calcular $d_L(\text{“gato”}, \text{“plato”})$.

1. $\texttt{gato} \xrightarrow{\text{Sustituir 'g' por 'p'}} \texttt{pato}$ (1 edición)
2. $\texttt{pato} \xrightarrow{\text{Insertar 'l'}} \texttt{plato}$ (1 edición)

La distancia total es $1 + 1 = 2$.

Ejemplo

$d_L(\text{“color”}, \text{“colour”}) = 1$ (una inserción de 'u').
$d_L(\text{“intención”}, \text{“ejecución”}) = 5$ (ej. i $\to$ e, n $\to$ j, t $\to$ e, e $\to$ c, sión $\to$ cución).

Observación

La distancia de Levenshtein se calcula eficientemente usando programación dinámica. Es fundamental para la corrección ortográfica, la bioinformática (alineamiento de secuencias) y la detección de plagio.

Métricas Textuales: Distancia de Levenshtein Implementación en Python

```python
import numpy as np

def levenshtein_distance(s1, s2):
    # Crear la matriz de distancias
    rows, cols = len(s1) + 1, len(s2) + 1
    dist_matrix = np.zeros((rows, cols), dtype=int)

    # Inicializar la primera fila y columna
    for i in range(rows):
        dist_matrix[i, 0] = i
    for j in range(cols):
        dist_matrix[0, j] = j

    # Rellenar la matriz usando programación dinámica
    for j in range(1, cols):
        for i in range(1, rows):
            cost = 0 if s1[i-1] == s2[j-1] else 1
            dist_matrix[i, j] = min(
                dist_matrix[i-1, j] + 1,         # Eliminación
                dist_matrix[i, j-1] + 1,         # Inserción
                dist_matrix[i-1, j-1] + cost     # Sustitución
            )

    return dist_matrix[rows-1, cols-1]
# Ejemplo de uso:
print(f"Distancia entre 'gato' y 'plato': {levenshtein_distance('gato', 'plato')}")
# Salida esperada: Distancia entre 'gato' y 'plato': 2
```

Resumen y Ejercicios

Cuadro Comparativo de Métricas

Métrica	Tipo de Dato Principal	Idea Clave
Minkowski (Euclidiana, Manhattan)	Vectores Numéricos	Mide distancia en espacio geométrico
Hamming	Secuencias de longitud fija	Cuenta posiciones diferentes
Jaccard	Conjuntos	Mide disimilitud por intersección/unión
Levenshtein	Cadenas (texto)	Mínimo de ediciones (insertar/borrar/sustituir)

Actividad (Ejercicios Rápidos)

1. Calcule la distancia de Hamming entre "casa" y "calle". ¿Es posible? ¿Por qué?
2. Calcule la distancia de Jaccard entre los conjuntos de letras $A = \{\text{a,b,c}\}$ y $B = \{\text{b,c,d,e}\}$.
3. ¿Cuál es la distancia de Levenshtein entre "sábado" y "sabado"?

Aplicación: Caso de Estudio Búsqueda de Películas Similares

Actividad (Encontrando películas similares en TMDB)

Contexto: Usando el archivo `tmdb_5000_movies.csv` de Kaggle, queremos encontrar las 3 películas más similares a *"Star Trek Beyond"*. La "similitud" puede definirse de muchas maneras, lo que requiere diferentes métricas.

Enfoques Posibles:

1. **Basado en Géneros:**
   - Representar cada película como un **conjunto de sus géneros**.
   - Calcular la **Distancia de Jaccard** entre "Star Trek Beyond" y todas las demás películas[1].
   - Las 3 películas con la menor distancia de Jaccard son las más similares en género.
2. **Basado en Sinopsis (Overview):**
   - Calcular la **Distancia de Levenshtein** entre el 'overview' de "Star Trek Beyond" y el de todas las demás.
   - Este método es computacionalmente **muy costoso** para un dataset grande.
   - Una alternativa más eficiente (que veremos más adelante) es usar técnicas de NLP como TF-IDF y la Similaridad del Coseno.
3. **Basado en Atributos Numéricos:**
   - Seleccionar variables como 'budget', 'revenue', 'popularity'.
   - **Estandarizar** estas variables.
   - Calcular la **Distancia Euclidiana** entre el vector de atributos de "Star Trek Beyond" y el de las demás.

[1] Debido al tamaño del dataset, se recomienda trabajar con una muestra aleatoria (ej. 100 películas) para probar los métodos de forma más ágil.

Fuentes principales:

- **Marc Peter Deisenroth, A. Aldo Faisal, y Cheng Soon Ong** (2020) *Mathematics for Machine Learning*. Cambridge University Press. Disponible en `https://mml-book.github.io/`
- **Daniel Peña** (2002) *Análisis de datos multivariantes.* McGraw-Hill
- **Avrim Blum, John Hopcroft y Ravindran Kannan** (2018) *Foundations of Data Science*. Cambridge University Press Disponible en `https://www.cs.cornell.edu/jeh/book.pdf`